Student Skill Development Management System

ASSIGNMENT PROBLEM REPORT

Submitted in the partial fulfilment for the Course of

CSA4305& Internet programming

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND BIOSCIENCE

Submitted by

MATHUSRI P(192311363)

POOJASREE B(192411091)

VENNA MEENAKSHI REDDY(192471048)

AMRITA ANAIKUMAR(192741035)

SRIJITHA S(192471024)

Under the Supervision of Guide Name

Dr.SUDHAGAR D

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

COURSE ASSIGNMENT

A. Assignment Information

Particular	Details
Department	CSE
Programme	B.Tech/B.E
Course Code & Course Name	CSA4305 – INTERNET PROGRAMMING
Academic Year / Batch	2026-27
Faculty Name	Dr.D.Sudhagar
Assignment Title	Student Skill Development Management System
Date of Issue	01.09.2026
Date of Submission	03.09.2026
Maximum Marks	100
Course Outcome(s) – CO	CO1: Develop visually appealing web pages using CSS for layout and style, and create dynamic, interactive functionality with JavaScript. CO2: Build dynamic web applications using XML, XSLT, and JSP within the MVC paradigm for efficient data handling and presentation.
Bloom's Taxonomy Level	L4 – Analyze / L5 – Evaluate / L6 – Create
SDG Mapping (SDG 1-17)	SDG 4 – Quality Education; SDG 8 – Decent Work and Economic Growth; SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Supports systematic tracking of student skills, certifications, projects, training and employability readiness for institutions and industry-oriented education.

B. Assignment Problem / Challenge

Design and develop a Student Skill Development Management System that enables students to create skill profiles, record certifications and projects, take skill assessments, identify competency gaps, access suitable learning activities, and monitor progress over time. The system should provide faculty/administrators with dashboards and analytics to evaluate skill development, recommend interventions and support employability-oriented decision-making. Students shall apply course concepts, analyze requirements and constraints, use modern tools, interpret results and justify design decisions.

C. Problem Statement

Educational institutions often maintain student academic records separately from information about programming skills, communication skills, certifications, projects, workshops, internships and other competencies. This makes it difficult to obtain a unified view of student skill development, identify gaps,

recommend suitable learning activities and measure improvement. Design a Student Skill Development Management System that provides a centralized platform for students, faculty/mentors and administrators.

The proposed system shall support skill-profile creation, skill categorization, self-assessment and/or test-based assessment, evidence upload, certification and project tracking, learning-plan or course recommendations, progress monitoring, mentor feedback, alerts and analytical reports. The solution should help stakeholders make evidence-based decisions about student development while protecting personal information.

D. Requirements and Constraints

- Functional requirements: student registration/login, profile management, skill entry, skill-level assessment, certification/project/internship records, learning-resource tracking, mentor feedback, goal setting, notifications and report generation.
- Performance requirements: responsive interaction, efficient retrieval of student skill records, reliable dashboard generation and timely updates.
- Data requirements: maintain structured information for students, skills, proficiency levels, assessments, evidence, learning activities, mentors, feedback and progress history.
- Technical constraints: use an approved programming language/framework, database and development environment; design the system using modular and maintainable components.
- Security/privacy: role-based access, authentication, authorization, input validation and controlled access to student records and uploaded evidence.
- Reliability: preserve assessment history and avoid accidental loss, duplication or inconsistent skill records.
- User requirements: provide simple workflows suitable for students, faculty/mentors and administrators with appropriate dashboards.
- Cost and time: develop the solution within the available academic resources and project schedule.
- Sustainability and scalability: support incremental addition of skills, learning resources and users without major redesign.

1. Problem Statement and Problem Formulation

1.1 Problem Statement

Educational institutions often store student academic records separately from programming skills, communication skills, certifications, projects, internships and other competencies. Because these details are fragmented, students may not have a single place to understand their current abilities, while faculty and mentors may find it difficult to identify competency gaps and recommend suitable learning activities. The proposed Student Skill Development Management System addresses this problem through a centralized web-based platform that connects student profiles, skills, assessments, evidence and learning plans in one interface.

The system is designed for students, faculty/mentors and administrators. Students can maintain their skill profile, record certifications and projects, take assessments and view recommended learning activities. Faculty or administrators can review skill progress, identify gaps and use analytics for development-oriented decisions.

1.2 Problem Formulation

- Input: student profile, skills, current skill levels, target levels, assessment answers, certifications, projects and learning progress.
- Processing: validation, score calculation, competency-gap identification, recommendation mapping and progress calculation.
- Output: updated skill profile, competency-gap report, learning recommendations, progress indicators and analytics.
- Goal: provide one unified, simple and interactive platform for evidence-based skill development.

2. Objective and Expected Outcomes

2.1 Objectives

- Develop a centralized web application for student skill-development management.
- Create an interactive student profile containing skills, certifications and projects.
- Provide skill assessment functionality and calculate competency levels.
- Identify gaps between current and target skill levels.
- Recommend suitable learning activities based on identified skill gaps.
- Track learning progress and display analytics in a user-friendly dashboard.
- Apply HTML5, CSS3, JavaScript, DOM, LocalStorage and event-handling concepts from Internet Programming.

2.2 Expected Outcomes

- A complete student skill profile with structured evidence.
- Clear identification of strengths and areas needing improvement.
- Automatic recommendations for learning activities.
- Visual progress monitoring using dashboards and analytics.
- Better organization of certification and project information.
- A maintainable front-end web solution demonstrating course concepts.

3. Requirements, Constraints and Assumptions

3.1 Functional Requirements

ID	Functional Requirement
FR1	Create and update student profile.
FR2	Add, edit and remove skills with category and level.
FR3	Store certifications and project portfolio entries.
FR4	Conduct skill assessments and calculate scores.
FR5	Perform competency gap analysis.
FR6	Generate learning recommendations from identified gaps.
FR7	Track learning-plan progress.
FR8	Display analytics such as completion and skill status.

3.2 Non-Functional Requirements

- Usability: simple navigation, readable labels and clear feedback.
- Performance: instant client-side interactions for common operations.
- Reliability: consistent storage and retrieval of student records.
- Security and privacy: sensitive information should not be exposed to unauthorized users.
- Maintainability: modular files and reusable JavaScript functions.
- Portability: the application should work in modern web browsers.

3.3 Hardware and Software Requirements

Category	**Requirement**
Processor	Dual-core processor or above
RAM	4 GB minimum; 8 GB recommended
Storage	At least 500 MB free for project files and tools
Operating System	Windows 10/11 or equivalent
Browser	Google Chrome, Microsoft Edge or Mozilla Firefox
Editor	Visual Studio Code / equivalent web editor
Technologies	HTML5, CSS3, JavaScript, Web Storage API
Optional Server	Local development server such as Live Server or Apache Tomcat when JSP is integrated

3.4 Constraints

- The basic implementation is primarily client-side and uses LocalStorage, so data is stored only in the browser used for testing.
- Recommendation quality depends on predefined skill-to-learning mappings.
- The prototype does not replace a full institutional student-information system.
- Large-scale multi-user deployment would require a backend database and authentication service.

3.5 Assumptions

- Students enter valid and updated skill information.
- Skill levels use a consistent scale, for example 1–5.
- The browser supports HTML5, modern CSS3, JavaScript and LocalStorage.
- The learning-plan catalog contains relevant resources for supported skills.

4. Application of Relevant Course Knowledge / Concepts

HTML5

HTML5 is used to structure the application pages, forms, navigation bars, cards, tables and dashboard sections. Semantic elements such as header, nav, main, section and footer improve organization.

CSS3

CSS3 is used for layouts, spacing, typography, responsive design, cards, buttons, progress indicators and visual styling. Flexbox and Grid can be used to create responsive dashboard layouts.

JavaScript

JavaScript provides the application logic for adding skills, scoring assessments, calculating gaps, generating recommendations and updating the interface without reloading the page.

DOM Manipulation

The Document Object Model is used to create, modify and remove elements dynamically. For example, newly added skills can immediately appear in a skill table or card section.

LocalStorage

LocalStorage provides simple browser-side persistence for the prototype. Student details, skills, certifications, projects and progress can be serialized using JSON and restored when the page is reopened.

Event Handling

Event listeners respond to form submissions, button clicks, input changes, filter actions and assessment completion. This makes the system interactive and responsive.

5. Design / Proposed Solution / Methodology

5.1 Proposed Solution

The proposed solution is a browser-based Student Skill Development Management System with a dashboard-driven interface. A student enters profile information, creates a skill portfolio, adds certifications and projects, completes assessments, reviews competency gaps and follows a learning plan. The same underlying student data is reused across modules so that the dashboard, analytics and recommendation sections stay synchronized.

Module	Primary Function
Dashboard	Shows overall profile summary, skill count, progress and alerts.
Student Profile	Stores name, register number, department and basic profile details.
Skill Management	Adds, updates and displays skills, categories and levels.
Skill Assessment	Presents questions and calculates assessment scores.
Certification Module	Stores certification title, issuer, date and evidence details.
Project Portfolio	Maintains project title, technology, description and completion status.
Learning Plan	Maps skill gaps to recommended learning activities and tracks completion.
Analytics Dashboard	Displays progress, competency gaps and summary metrics.

5.2 Methodology

1. Analyze assignment requirements and identify user roles and data items.
2. Design page flow and module relationships.
3. Create HTML5 structure and CSS3 styles for a consistent interface.
4. Implement JavaScript data models and event handlers.
5. Use LocalStorage as the prototype persistence layer.
6. Implement assessment scoring, competency-gap calculation and recommendation rules.
7. Create dashboard and analytics views from the same stored dataset.
8. Execute test cases and validate expected outputs.
9. Review trade-offs and identify future improvements such as backend database integration.

6. Algorithm / Pseudocode / Flowchart

6.1 Algorithm: Add Skill

10. Start.
11. Read skill name, category, current level and target level from the form.
12. Check that required fields are not empty and levels are within the valid range.
13. Create a skill object with a unique identifier.
14. Read the existing skill list from LocalStorage.
15. Append the new skill object to the list.
16. Save the updated list back to LocalStorage.
17. Refresh the skill display and dashboard indicators.
18. Show a success message.
19. Stop.

6.2 Pseudocode: Skill Assessment

```
START
  score ← 0
  FOR each question IN assessment
     READ selectedAnswer
     IF selectedAnswer = correctAnswer THEN
        score ← score + marks
     END IF
  END FOR
  percentage ← (score / maximumMarks) × 100
  competency ← MAP_PERCENTAGE_TO_LEVEL(percentage)
  DISPLAY score, percentage, competency
END
```

6.3 Pseudocode: Competency Gap Analysis

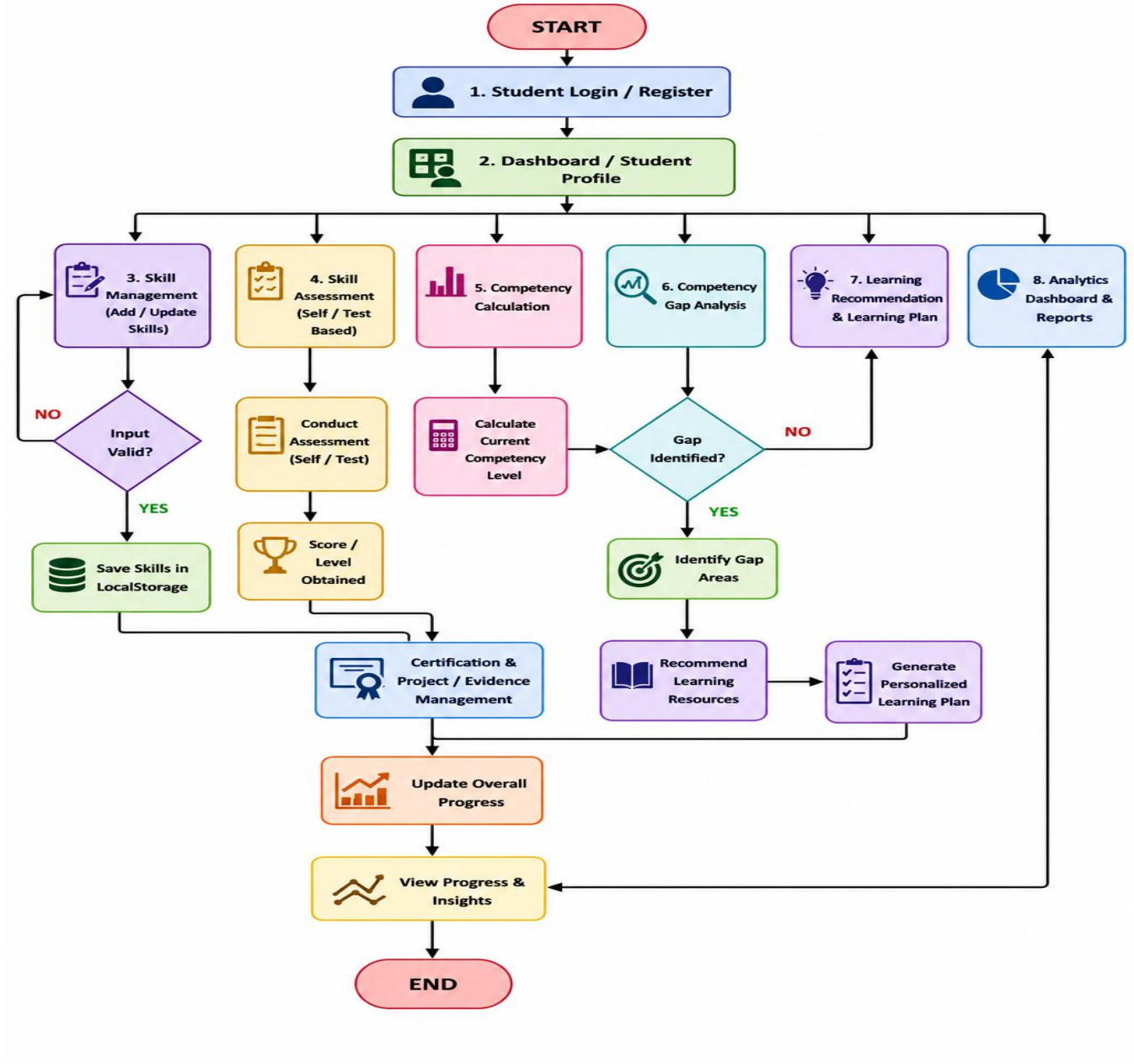
```
START
  FOR each skill IN studentSkills
     gap ← skill.targetLevel - skill.currentLevel
     IF gap <= 0 THEN
        status ← "Target Achieved"
     ELSE
        status ← "Skill Gap"
     END IF
```

```
    STORE gap and status
  END FOR
  DISPLAY results
END
```

6.4 Pseudocode: Learning Recommendation

```
START
  FOR each skillGap IN identifiedGaps
    resources ← FIND_RESOURCES(skillGap.skillName)
    FOR each resource IN resources
      IF resource.level >= skillGap.requiredLevel THEN
        ADD resource TO recommendations
      END IF
    END FOR
  END FOR
  DISPLAY recommendations
END
```

6.5 Flowchart



7. Implementation / Source Code and Environment / Tools Used

Technologies Used

Technology	Use in the Project
HTML5	Page structure, forms and semantic sections
CSS3	Layout, responsive design and visual styling
JavaScript ES6+	Application logic, calculations and interaction
DOM API	Dynamic page updates and element manipulation
LocalStorage API	Prototype data persistence
VS Code	Source-code editing and project management
Chrome / Edge	Execution and testing

Project Structure

```
StudentSkillDevelopmentManagementSystem/
├── index.html
├── dashboard.html
├── css/
│   └── style.css
├── js/
│   ├── app.js
│   ├── skills.js
│   ├── assessment.js
│   ├── recommendations.js
│   └── analytics.js
├── data/
│   └── learning-resources.json
└── assets/
    └── images/
```

Main Modules

- Dashboard
- Student Profile
- Skill Management
- Skill Assessment
- Certification Module
- Project Portfolio
- Learning Plan
- Analytics Dashboard

Representative Source Code

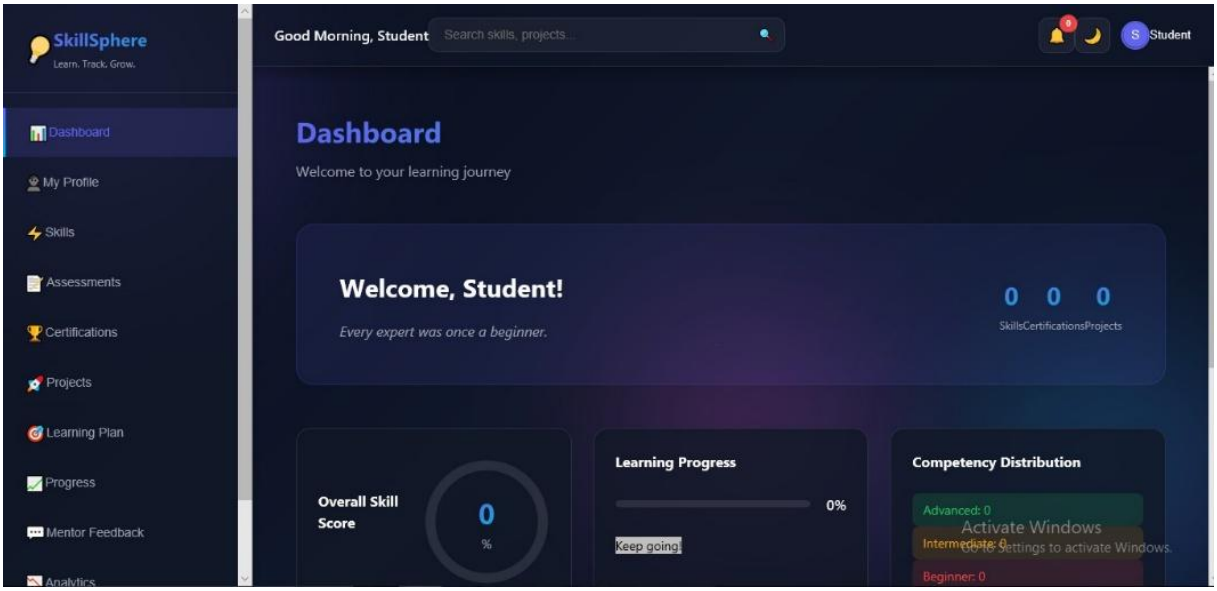
A compact implementation pattern is shown below. Replace the sample with the exact project code before final submission when complete source code is required.

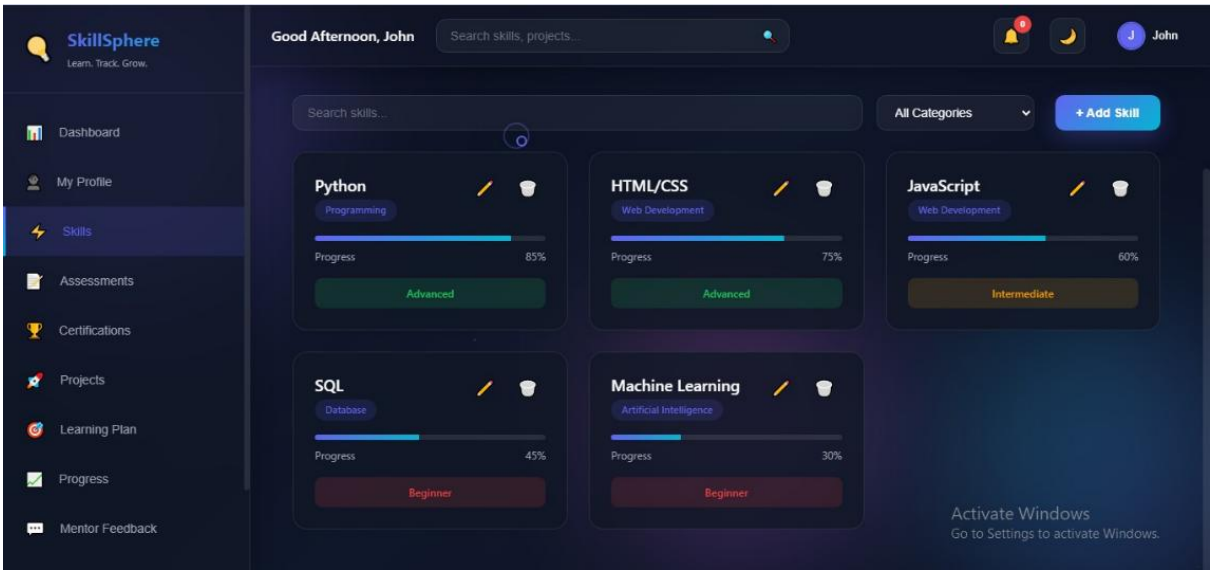
```
function addSkill() {
  const skill = {
    name: document.getElementById("skillName").value.trim(),
    category: document.getElementById("skillCategory").value,
    currentLevel: Number(document.getElementById("currentLevel").value),
    targetLevel: Number(document.getElementById("targetLevel").value)
  };

  const skills = JSON.parse(localStorage.getItem("skills") || "[]");
  skills.push(skill);
  localStorage.setItem("skills", JSON.stringify(skills));
  renderSkills();
}
```

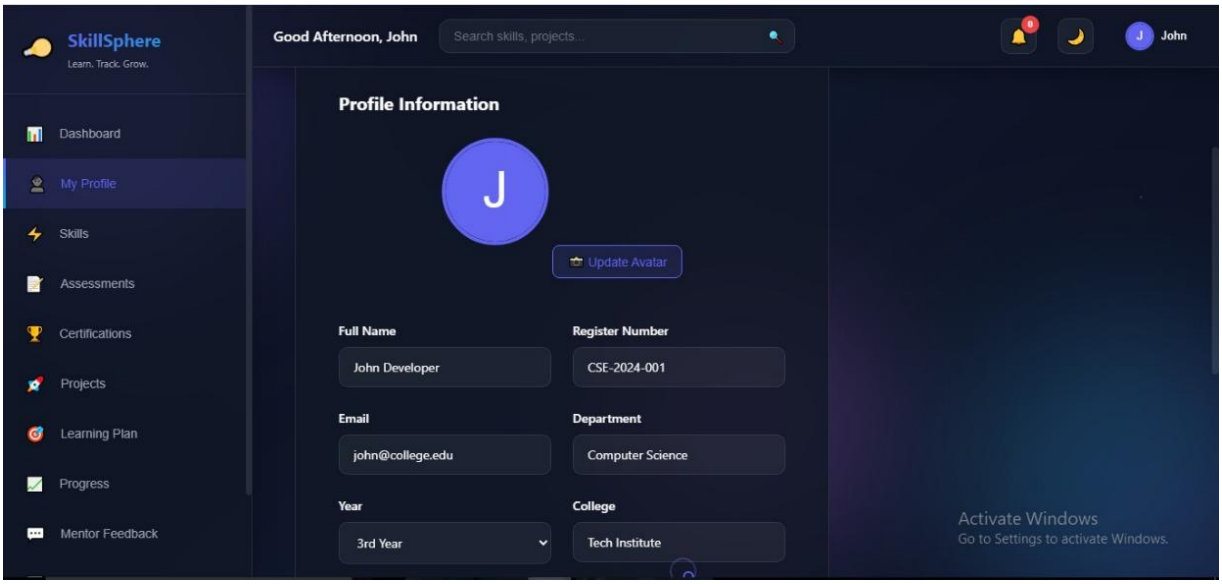
8. Test Cases and Expected/Actual Results

ID	Test Case	Input / Action	Expected Result	Actual Result
TC01	Add valid skill	Enter Java, Technical, Current=2, Target=4	Skill saved and displayed	To be recorded
TC02	Empty skill field	Leave skill name blank	Validation message displayed	To be recorded
TC03	Invalid level	Enter level 6	Invalid input rejected	To be recorded
TC04	Assessment scoring	Submit valid answers	Score and competency displayed	To be recorded
TC05	Gap analysis	Current 2, Target 4	Gap shown as 2	To be recorded
TC06	Recommendation	Gap exists for Java	Java-related learning resources displayed	To be recorded
TC07	Certification entry	Enter valid certificate details	Certificate appears in module	To be recorded
TC08	Project entry	Add a project	Project appears in portfolio	To be recorded
TC09	Progress update	Mark learning activity complete	Progress indicator increases	To be recorded
TC10	Analytics	Open dashboard after updates	Metrics match stored dataset	To be recorded
TC11	LocalStorage persistence	Reload browser page	Previously saved prototype data remains	To be recorded
TC12	Responsive UI	Resize browser window	Layout remains readable and usable	To be recorded

9. Execution Screenshots / Outputs







10. Results and Validation

Sample Skill Data

Student	Skill	Category	Current	Target	Status
S001	Java	Programming	2	4	Gap 2
S001	HTML/CSS	Web	4	4	Target Achieved
S001	JavaScript	Web	3	5	Gap 2
S001	Communication	Soft Skill	2	3	Gap 1
S001	SQL	Database	3	4	Gap 1

Competency Gap Analysis

For each skill, the competency gap is computed as Target Level − Current Level. Positive values indicate a need for improvement. Zero or negative values indicate that the student has reached or exceeded the selected target.

Skill	Current	Target	Gap	Interpretation
Java	2	4	2	Needs focused learning
HTML/CSS	4	4	0	Target achieved
JavaScript	3	5	2	Needs focused learning
Communication	2	3	1	Needs improvement
SQL	3	4	1	Needs improvement

Overall Progress Calculation

A simple overall progress measure can be calculated from the average achievement ratio across tracked skills:

Progress % = [Σ (Current Level / Target Level) / Number of Skills] × 100

Using the sample dataset: (2/4 + 4/4 + 3/5 + 2/3 + 3/4) / 5 × 100 ≈ 68.3%. This is an illustrative value for validating the calculation logic and should be replaced by the actual value generated by the final dataset.

11. Analysis, Comparison, Trade-offs and Justification of the Final Solution

Analysis

The system centralizes student skill-development information and links the modules through a shared dataset. Because skill levels, assessments and learning activities are related, a change in one module can be reflected in the dashboard and analytics. This reduces manual calculation and gives students a clearer picture of development needs.

Comparison

Approach	Advantages	Limitations
Manual records / spreadsheets	Easy to begin, familiar to users	Duplicate data, weak integration and limited interaction
Static website	Simple UI and low setup effort	No persistent structured skill workflow or dynamic analytics
JavaScript + LocalStorage prototype	Interactive, quick to develop and demonstrates course concepts	Browser-local data, limited multi-user support
Full web app with backend database	Centralized multi-user storage, authentication and stronger reporting	Higher development and deployment complexity

Trade-offs

- LocalStorage gives fast prototype development but is not suitable as a secure institutional database.
- Client-side processing provides immediate interaction but needs server-side validation in a production system.
- Rule-based recommendations are simple and explainable, but more advanced personalization would require more data and algorithms.
- A modular interface improves maintainability but requires careful synchronization of shared data.

Justification

The final solution is justified for the assignment because it directly addresses the stated problem while applying HTML5, CSS3, JavaScript, DOM manipulation, LocalStorage and event handling. It offers a clear demonstration of interactive web programming, keeps modules connected through one dataset, and can later be upgraded to a database-backed architecture.

12. Broader Considerations / SDG Relevance

SDG 4 – Quality Education

The system supports continuous learning by helping students assess their skills, identify gaps, follow learning plans and monitor improvement.

SDG 8 – Decent Work and Economic Growth

By organizing job-relevant skills, certifications and projects, the system can strengthen employability-oriented development and help students recognize areas that require further preparation.

SDG 9 – Industry, Innovation and Infrastructure

The project demonstrates a digital infrastructure approach for structured skill development and institutional analytics. It can serve as a base for future integration with larger educational systems.

13. Conclusion, Limitations and Possible Improvements

13.1 Conclusion

The Student Skill Development Management System provides a structured and interactive environment for maintaining student profiles, skills, assessments, certifications, projects, learning plans and analytics. The solution demonstrates the practical application of core Internet Programming concepts and provides a foundation for evidence-based skill development.

13.2 Limitations

- LocalStorage is browser-specific and does not provide centralized multi-user storage.
- The prototype cannot securely authenticate multiple users without a backend.
- Recommendations depend on predefined mappings between skills and learning resources.
- Assessment quality depends on the selected questions and competency scale.

13.3 Possible Improvements

- Add MySQL or another backend database for centralized storage.
- Implement secure login and role-based access.
- Add JSP/Servlet or REST APIs for server-side processing.
- Add richer charts and historical progress analytics.
- Add automated certificate verification and more detailed competency frameworks.
- Integrate institutional learning resources and notifications.

14. Individual Contribution of Group Members

Member	Individual Contribution
MATHUSRI P (192311363)	Requirement analysis, problem statement, problem formulation and overall system planning.
POOJASREE B (192411091)	HTML5 and CSS3 implementation, page layouts, navigation and user interface design.
VENNA MEENAKSHI REDDY (192471048)	JavaScript implementation, DOM manipulation, event handling and LocalStorage functionality.
AMRITA ANAIKUMAR (192741035)	Skill assessment, competency gap analysis, learning recommendation and analytics modules.
SRIJITHA S (192471024)	Testing, validation, execution screenshots, result analysis, documentation and report preparation.

15. References

1. MDN Web Docs, “HTML: HyperText Markup Language,” Mozilla Developer Network.
2. MDN Web Docs, “CSS: Cascading Style Sheets,” Mozilla Developer Network.
3. MDN Web Docs, “JavaScript Guide,” Mozilla Developer Network.
4. MDN Web Docs, “Document Object Model (DOM),” Mozilla Developer Network.
5. MDN Web Docs, “Window: localStorage property,” Web Storage API documentation.
6. WHATWG, “HTML Living Standard,” Web Hypertext Application Technology Working Group.
7. W3C, “Cascading Style Sheets (CSS) Specifications,” World Wide Web Consortium.
8. W3C, “DOM Standard,” World Wide Web Consortium.
9. Apache Tomcat Documentation, Apache Software Foundation, for optional JSP/Servlet deployment.
10. United Nations, “Sustainable Development Goals,” especially SDG 4, SDG 8, and SDG 9.
11. Course notes and laboratory material for CSA4305 – Internet Programming.
12. Project dataset and learning-resource records created for the Student Skill Development Management System prototype.